

---

# Introducción al *deep learning*

---

PID\_00290601

Jordi Casas Roma

---

Tiempo mínimo de dedicación recomendado: 2 horas

---



---

Universitat  
Oberta  
de Catalunya

---

**Jordi Casas Roma**

Doctor en Informática (Universitat Autònoma de Barcelona, UAB, 2014), Máster en Inteligencia Artificial Avanzada (Universidad Nacional de Educación a Distancia, UNED, 2011) y Licenciado en Informática (Universitat Autònoma de Barcelona, UAB, 2002). Actualmente, es profesor agregado de la Facultad de Informática, Multimedia y Telecomunicaciones de la Universitat Oberta de Catalunya (UOC). Sus actividades docentes se concentran principalmente en minería de datos, aprendizaje automático, *big data* y minería de gráficos. Es el director del máster en Ciencia de datos (Data Science) de la Universitat Oberta de Catalunya (UOC). Sus principales ámbitos de investigación incluyen la inteligencia artificial, el aprendizaje automático, minería de grafos y la privacidad de datos. Desde 2020 es líder de grupo en ADaS Lab (Applied Data Science Lab) donde investiga la aplicación de algoritmos de inteligencia artificial a problemas del mundo real.

El encargo y la creación de este recurso de aprendizaje UOC han sido coordinados por el profesor: Jordi Casas Roma

**Cómo citar este recurso de aprendizaje con el estilo Harvard:**

Casas Roma, J. (2023) *Introducción al deep learning*. [Recurso de aprendizaje textual]. 1ª edición. Barcelona: Fundació Universitat Oberta de Catalunya (FUOC).

Primera edición: febrero 2024

© de esta edición, Fundació Universitat Oberta de Catalunya (FUOC)

Av. Tibidabo, 39-43, 08035 Barcelona

Autoría: Jordi Casas Roma

Producción: FUOC

Todos los derechos reservados

*Ninguna parte de esta publicación, incluyendo el diseño general y la cubierta, puede ser copiada, reproducida, almacenada o transmitida de ninguna forma ni por ningún medio, sea este eléctrico, químico, mecánico, óptico, grabación, fotocopia, o cualquier otro, sin la previa autorización escrita de los titulares de los derechos.*

# Índice

|                                                             |    |
|-------------------------------------------------------------|----|
| <b>Introducción</b> .....                                   | 5  |
| <b>Objetivos</b> .....                                      | 6  |
| <b>1. Introducción y contextualización</b> .....            | 7  |
| 1.1. ¿Qué es <i>deep learning</i> ? .....                   | 7  |
| 1.2. Contextualización de las redes neuronales .....        | 9  |
| <b>2. Conceptos básicos de aprendizaje automático</b> ..... | 11 |
| 2.1. Tipología de métodos .....                             | 11 |
| 2.2. Tipología de tareas .....                              | 12 |
| 2.2.1. Clasificación .....                                  | 12 |
| 2.2.2. Regresión .....                                      | 13 |
| 2.2.3. Agrupamiento .....                                   | 14 |
| 2.3. Preprocesamiento de datos .....                        | 15 |
| 2.3.1. Datos estructurados .....                            | 15 |
| 2.3.2. Imágenes .....                                       | 17 |
| 2.4. Datos de entrenamiento y test .....                    | 17 |
| 2.4.1. <i>k-fold cross validation</i> .....                 | 18 |
| 2.5. Evaluación de modelos .....                            | 19 |
| 2.5.1. Modelos de clasificación .....                       | 19 |
| 2.5.2. Modelos de regresión .....                           | 23 |
| <b>3. Apéndice A: Notación</b> .....                        | 25 |
| <b>Resumen</b> .....                                        | 26 |
| <b>Glosario</b> .....                                       | 27 |
| <b>Bibliografía</b> .....                                   | 28 |

## Introducción

Este módulo didáctico constituye la introducción al aprendizaje profundo (*deep learning*, en inglés), en general, y a las redes neuronales, en particular.

Iniciaremos este módulo didáctico definiendo qué es el aprendizaje profundo y cuáles son sus principales características y diferencias con otros tipos de aprendizaje automático. Complementaremos esta primera parte con un breve repaso histórico sobre las redes neuronales, que constituyen la base de los modelos de aprendizaje profundo.

A continuación, se repasan algunos conceptos básicos sobre aprendizaje automático que son necesarios para comprender y profundizar en los modelos de aprendizaje profundo. En concreto, revisaremos los conceptos relacionados con la tipología de métodos y tareas en el aprendizaje automático, así como los principios de preprocesamiento de datos, generación de los conjuntos de entrenamiento y test, y métricas de evaluación de resultados.

## Objetivos

En los materiales didácticos de este módulo encontraremos las herramientas indispensables para asimilar los siguientes objetivos:

1. Entender los principios fundamentales del aprendizaje profundo (o *deep learning*, en inglés).
2. Conocer la tipología básica de métodos y tareas en aprendizaje automático.
3. Conocer los fundamentos de preprocesamiento de datos, especialmente en el caso de imágenes.
4. Comprender como se generan los conjuntos de datos de entrenamiento y test.
5. Conocer las principales métricas de evaluación de resultados.

## 1. Introducción y contextualización

En este capítulo discutiremos, brevemente, qué entendemos por *deep learning* y haremos un breve repaso histórico para revisar los hechos que nos han permitido llegar hasta el momento actual.

### 1.1. ¿Qué es *deep learning*?

Desde los inicios de la inteligencia artificial (IA) se ha soñado con máquinas capaces de «pensar» de forma similar a como lo hacemos los seres humanos. Aunque aún estamos lejos de conseguir este ambicioso objetivo, hace ya unas décadas se consiguieron superar importantes desafíos en ámbitos o problemas concretos. Por ejemplo, en 1997, la computadora Deep Blue de IBM fue capaz de ganar al ajedrez al campeón mundial Garry Kasparov. Aunque saber jugar correctamente al ajedrez es un problema muy complejo para los seres humanos, este se rige por una serie de normas estrictas y en un entorno muy delimitado y perfectamente especificado. Esto hace que lo que es una tarea compleja (o muy compleja) para una persona, sea una tarea «fácil» de implementar en un ordenador. Los ordenadores, por regla general, se manejan bien en los escenarios formales. Es decir, escenarios cerrados, regidos por reglas y con un conjunto de actores finito. A partir de aquí, los ordenadores tienen herramientas para inferir situaciones futuras a partir de la situación actual y las reglas que rigen el escenario.

En muchos de estos casos, las reglas y el conocimiento son introducidos en el sistema en el momento de su programación. Es decir, es el programador el encargado de «transferir» el conocimiento al modelo, quién después será capaz de inferir nuevo conocimiento a partir de las reglas. Un caso típico son los sistemas basados en reglas (*Rule-Based Systems*, RBS).

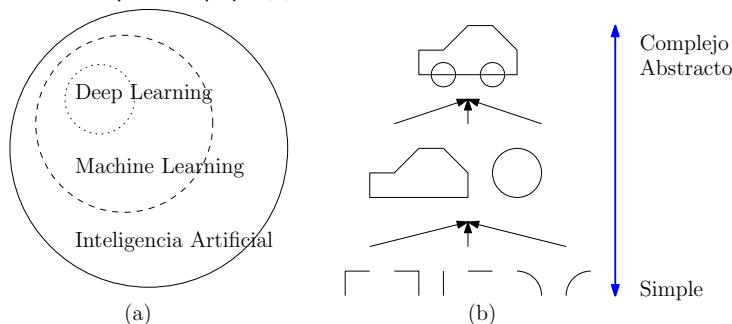
Por el contrario, algunas de las acciones más elementales e intuitivas para los humanos, como por ejemplo reconocer una persona, un animal o un objeto, pueden ser tareas bastante complejas de resolver para un ordenador. La principal dificultad radica en el hecho de que, en estos casos, no es fácil formalizar los conceptos abstractos que aparecen en estos escenarios. Definir formalmente los conceptos abstractos relacionados con estos escenarios es una tarea compleja. La representación de los conceptos deviene un punto clave en la implementación de modelos y, en el caso de los conceptos abstractos, la complejidad de la tarea dificulta enormemente que pueda ser implementado mediante los modelos de IA clásicos.

#### Lectura complementaria

Feng-Hsiung Hsu. «Behind Deep Blue: Building the Computer That Defeated the World Chess Champion». Princeton University Press, Princeton, NJ, USA, 2002.

Debido a la dificultad para introducir este tipo de conocimiento *a priori* en los modelos, se ha optado por utilizar **modelos que sean capaces de extraer su propio conocimiento a partir de los datos**. Esta área se ha denominado aprendizaje automático o *machine learning*. Existen multitud de técnicas y modelos que permiten aprender a partir de un conjunto de datos, como por ejemplo modelos de regresión, modelos estadísticos o las máquinas de vectores soporte (SVM, *Support Vector Machine*). En muchos casos, los modelos son capaces de aprender patrones a partir de los datos, pero **no son capaces de aprender conceptos abstractos y complejos a partir de estos datos**.

Figura 1. Relación del *deep learning* con otros conceptos (a) y ejemplo de jerarquía para la generación de conceptos complejos (b)



El concepto de *deep learning* (o aprendizaje profundo) aparece como un subconjunto dentro del aprendizaje automático (véase figura 1 (a)), donde se persigue crear **modelos que sean capaces de representar conceptos complejos y/o abstractos a partir de conceptos más sencillos**. Es decir, el modelo es capaz de crear de forma automática **una jerarquía de conceptos**, empezando por conceptos simples, e ir mezclando estos conceptos más simples para ir creando conceptos cada vez más complejos. Esto permite definir conceptos abstractos como composiciones de conceptos mucho más simples. La figura 1 (b) muestra un sencillo ejemplo de esta composición de conceptos complejos o abstractos a partir de conceptos mucho más simples y sencillos. En el ejemplo vemos como, a partir de líneas, cruces y arcos es posible definir el concepto de coche, especificándolo como una composición de estos elementos más simples. Cuando esta jerarquía de conceptos tiene múltiples capas, hablamos de la «profundidad» del modelo. Es aquí donde aparece el concepto de «aprendizaje profundo» o *deep learning*.

Aunque actualmente se asocia el concepto de *deep learning* con las redes neuronales, existen otros modelos de aprendizaje automático que también son capaces de componer conceptos a partir de una jerarquía de otros más simples. Aun así, en este texto nos centraremos en las distintas variedades de redes neuronales profundas, que actualmente representan el estado del arte en el *deep learning*.

## 1.2. Contextualización de las redes neuronales

Las redes neuronales artificiales (*Artificial Neural Networks* o ANN) son un conjunto de algoritmos inspirados en el mecanismo de comunicación de la neurona biológica. Han demostrado ser una buena aproximación a problemas donde el conocimiento es impreciso o variante en el tiempo. Su capacidad de aprender convierte a las redes neuronales en algoritmos adaptativos y elaborados a la vez.

El inicio de las redes neuronales se remonta a los años cuarenta, cuando McCulloch y Pitts (McCulloch & Pitts, 1943) propusieron un modelo para intentar explicar el funcionamiento del cerebro humano. Una década después, en los años cincuenta, se empezaron a implementar modelos de cálculo basados en el concepto del perceptrón (Rosenblatt, 1962). Posteriormente, Marvin Minsky demostró las limitaciones teóricas de los modelos existentes (Minsky & Papert, 1969), provocando un estancamiento de la investigación y aplicaciones relacionadas con las redes neuronales durante los años setenta y ochenta. Hasta que, en 1984, John Hopfield propuso el modelo de la «propagación hacia atrás» (*backpropagation*) (Hopfield, 1984), que permitió superar las limitaciones del perceptrón y, además, facilitó en gran medida la fase de entrenamiento de las redes.

Las grandes expectativas puestas en las redes neuronales durante las décadas de 1980 y 1990 provocaron una considerable desilusión en estos modelos, que no fueron capaces de cumplir las (altas) expectativas generadas en aquel momento. Aun así, la investigación y el avance en las redes neuronales continuó durante estos años, aunque sin una gran atención por parte de la comunidad científica, que se centró en otros modelos que parecían más prometedores en aquel entonces.

El resurgimiento de las redes neuronales, en este caso bajo la etiqueta de *deep learning*, se produjo a partir de mediados de la primera década de este siglo. En 2006, Hinton *et al.* (2006) presentaron un algoritmo que permitió entrenar redes *deep belief* de forma eficiente, terminando así con el período de dificultades relacionadas con el entrenamiento de redes. Otros autores desarrollaron métodos similares para otros tipos de datos, como Bengio *et al.* (2006). Fue a partir de este momento cuando se popularizó el término «deep learning», haciendo referencia, especialmente, al concepto de ser capaces de entrenar redes más profundas. Actualmente, las redes profundas son el estado del arte en muchos dominios, donde han superado ampliamente a los demás modelos o algoritmos de aprendizaje automático.

Es importante remarcar que el término moderno *deep learning* va más allá del concepto biológico que inspiró, inicialmente, a las redes neuronales para centrarse en el principio de los múltiples niveles de composición que forman los conceptos complejos y/o abstractos. Por lo tanto, actualmente la neuro-

### Lectura complementaria

W. S. McCulloch, W. Pitts. «A logical calculus of the ideas immanent in nervous activity». *Neurocomputing: Foundations of Research*, p. 15-27. MIT Press, 1943.

### Lectura complementaria

F. Rosenblatt. «Principles of neurodynamics; perceptrons and the theory of brain mechanisms». SpartanBooks, 1962.

### Lectura complementaria

M. Minsky, S. Papert. «Perceptrons: An Introduction to Computational Geometry». MIT Press, Cambridge, MA, USA, 1969.

### Lectura complementaria

J. J. Hopfield. «Neurons with graded response have collective computational properties like those of two-state neurons». *Proc. of the National Academy of Sciences*, Vol. 81(10), p. 3088-3092, 1984.

### Lectura complementaria

G. E. Hinton, S. Osindero, Y.-W. Teh. «A fast learning algorithm for deep belief nets». *NeuralComput.*, Vol. 18(7), p. 1527-1554, 2006.



ciencia se toma solo como una inspiración para los modelos de *deep learning*, pero en ningún caso como una «guía» para el desarrollo de nuevos modelos o arquitecturas. Simplemente, porque no tenemos, hoy en día, suficiente conocimiento de cómo trabaja nuestro cerebro a bajo nivel para poder entender el funcionamiento global de un conjunto de miles o millones de neuronas.

**Lectura  
complementaria**

Y. Bengio, P. Lamblin, D. Popovici, H. Larochelle.  
«Greedy layer-wise training of deep networks». In Proc. NIPS, p. 153-160, Cambridge, MA, USA, 2006. MIT Press.

## 2. Conceptos básicos de aprendizaje automático

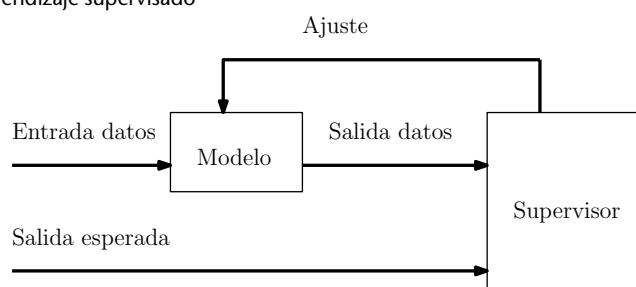
El aprendizaje automático (más conocido por su denominación en inglés, *Machine Learning*, ML) es el conjunto de métodos y algoritmos que permiten a una máquina aprender de manera automática en base a experiencias pasadas.

En este capítulo veremos algunos conceptos básicos de aprendizaje automático, en general, que son especialmente relevantes para el uso de redes neuronales artificiales. En este sentido, empezaremos viendo la tipología de métodos y tareas existentes en aprendizaje automático. A continuación, revisaremos algunas medidas básicas de preprocesamiento de datos y discutiremos muy brevemente la creación del conjunto de entrenamiento y test. Finalizaremos este capítulo revisando algunas medidas de evaluación de modelos que nos resultarán de gran utilidad posteriormente.

### 2.1. Tipología de métodos

Generalmente, un algoritmo de aprendizaje automático debe construir un modelo en base a un conjunto de datos de entrada que representan el conjunto de aprendizaje, lo que se conoce como *conjunto de entrenamiento*. Durante esta fase de aprendizaje, el algoritmo va comparando la salida de los modelos en construcción con la salida ideal que deberían tener estos modelos para ir ajustándolos y aumentando la precisión. Esta comparación forma la base del aprendizaje en sí y puede ser **supervisado** o **no supervisado**. En el aprendizaje supervisado (figura 2), hay un componente externo que compara los resultados obtenidos por el modelo con los resultados esperados por este, y proporciona retroalimentación al modelo para que vaya ajustándose. Para ello, pues, será necesario proporcionar el modelo con un conjunto de datos de entrenamiento que contenga tanto los datos de entrada como la salida esperada para cada uno de ellos.

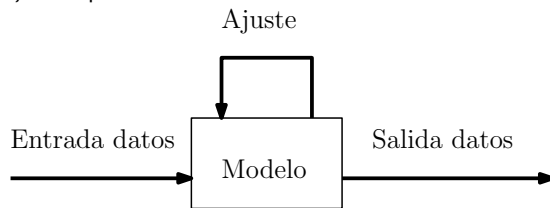
Figura 2. Aprendizaje supervisado



Todas ellas se basan en el paradigma del aprendizaje inductivo. La esencia de cada una de ellas es derivar inductivamente a partir de los **datos** (que representan la información del entrenamiento), un **modelo** (que representa el conocimiento) que tiene utilidad predictiva, es decir, que puede aplicarse a nuevos datos.

En el aprendizaje no supervisado (figura 3), el algoritmo de entrenamiento aprende sobre los propios datos de entrada, descubriendo y agrupando patrones, características, correlaciones, etc.

Figura 3. Aprendizaje no supervisado



## 2.2. Tipología de tareas

Según el objetivo de nuestro análisis, podemos distinguir entre tres grandes grupos de tareas, que revisaremos brevemente a continuación.

### 2.2.1. Clasificación

La clasificación (*classification*) es uno de los procesos cognitivos importantes, tanto en la vida cotidiana como en los negocios, donde podemos clasificar clientes, empleados, transacciones, tiendas, fábricas, dispositivos, documentos o cualquier otro tipo de instancias en un conjunto de clases o categorías predefinidas con anterioridad.

La tarea de clasificación consiste en asignar instancias de un dominio dado, descritas por un conjunto de atributos discretos o de valor continuo, a un conjunto de clases, que pueden ser consideradas valores de un atributo discreto seleccionado, generalmente denominado **clase**. Las etiquetas de clase correctas son, en general, desconocidas, pero se proporcionan para un subconjunto del dominio. Por lo tanto, queda claro que es necesario disponer de un subconjunto de datos correctamente etiquetado, que se usará para la construcción del modelo.

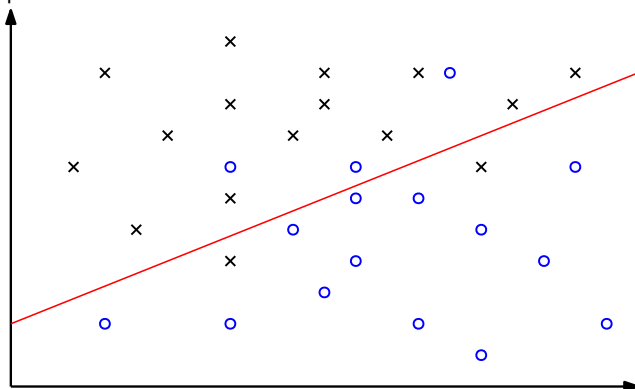
La función de clasificación puede verse como:

$$c : X \rightarrow C \quad (1)$$

donde  $c$  representa la función de clasificación,  $X$  el conjunto de atributos que forman una instancia y  $C$  la etiqueta de clase de dicha instancia.

Un tipo de clasificación particularmente simple, pero muy interesante y ampliamente estudiado, hace referencia a los problemas de clasificación binarios, es decir, problemas con un conjunto de datos pertenecientes a dos clases, i.e.  $C = \{0, 1\}$ . La figura 4 muestra un ejemplo de clasificación binaria, donde las cruces y los círculos representan elementos de dos clases, y se pretende dividir el espacio de tal forma que separe a la mayoría de elementos de clases diferentes.

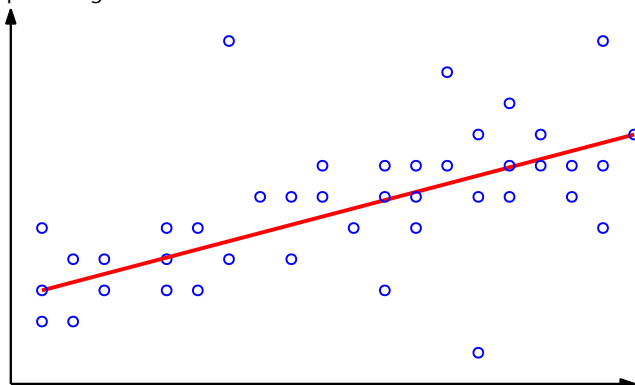
Figura 4. Ejemplo de clasificación



### 2.2.2. Regresión

Al igual que la clasificación, la regresión (*regresion*) es una tarea de aprendizaje inductivo que ha sido ampliamente estudiada y utilizada. Se puede definir, de forma informal, como un problema de «clasificación con clases continuas». Es decir, los modelos de regresión predicen valores numéricos en lugar de etiquetas de clase discretas. A veces también nos podemos referir a la regresión como «predicción numérica».

Figura 5. Ejemplo de regresión lineal



La tarea de regresión consiste en asignar valores numéricos a instancias de un dominio dado, descritos por un conjunto de atributos discretos o de valor continuo, como se muestra en la figura 5, donde los puntos representan los datos de aprendizaje y la línea representa la predicción sobre futuros eventos. Se supone que esta asignación se aproxima a alguna función objetivo,

generalmente desconocida, excepto para un subconjunto del dominio. Este subconjunto se puede utilizar para crear el modelo de regresión.

En este caso, la función de regresión se puede definir como:

$$f : X \rightarrow \mathbb{R} \quad (2)$$

donde  $f$  representa la función de regresión,  $X$  el conjunto de atributos que forman una instancia y  $\mathbb{R}$  un valor en el dominio de los números reales.

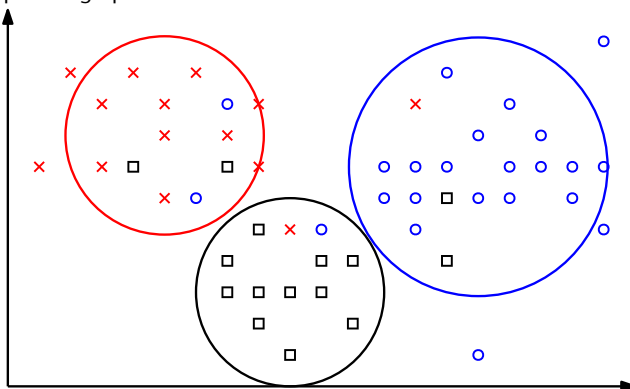
Es importante remarcar que una regresión no pretende devolver una predicción exacta sobre un evento futuro, sino una aproximación (como muestra la diferencia entre la línea y los puntos de la figura). Por lo general, datos más dispersos resultarán en predicciones menos ajustadas.

### 2.2.3. Agrupamiento

El agrupamiento (*clustering*) es una tarea de aprendizaje inductiva que, a diferencia de las tareas de clasificación y regresión, no dispone de una etiqueta de clase a predecir. Puede considerarse como un problema de clasificación, pero donde no existen un conjunto de clases predefinidas y estas se «descubren» de forma autónoma por el método o algoritmo de agrupamiento, basándose en patrones de similitud identificados en los datos.

La tarea de agrupamiento consiste en dividir un conjunto de instancias de un dominio dado, descrito por un número de atributos discretos o de valor continuo, en un conjunto de grupos (*clusters*) basándose en la similitud entre las instancias, y crear un modelo que puede asignar nuevas instancias a uno de estos grupos o *clusters*. La figura 6 muestra un ejemplo de agrupamiento, donde las cruces, los círculos y cuadros pertenecen a tres clases de elementos distintos que pretendemos agrupar.

Figura 6. Ejemplo de agrupamiento



Un proceso de agrupación puede proporcionar información útil sobre los patrones de similitud presentes en los datos, como por ejemplo, segmentación de clientes o creación de catálogos de documentos. Otra de las principales utilidades del agrupamiento es la detección de anomalías. En este caso, el método de agrupamiento permite distinguir instancias con un patrón distinto a las demás instancias «normales» del conjunto de datos, facilitando la detección de anomalías y posibilitando la generación de alertas automáticas para nuevas instancias que no corresponden a ningún *cluster* existente.

La función de agrupamiento o *clustering* se puede modelar mediante:

$$h : X \rightarrow C_h \quad (3)$$

donde  $h$  representa la función de agrupamiento,  $X$  el conjunto de atributos que forman una instancia y  $C_h$  un conjunto de grupos o *clusters*. Aunque esta definición se parece mucho a la tarea de clasificación, una diferencia aparentemente pequeña, pero muy importante, consistente en que el conjunto de «clases» no está predeterminado ni es conocido *a priori*, sino que se identifica como parte de la creación del modelo.

### 2.3. Preprocesamiento de datos

Las redes neuronales artificiales, como el resto de modelos de aprendizaje automático, requieren los datos de entrada en un formato específico para poder trabajar de forma correcta.

Veamos las principales características de los datos de entrada de una red neuronal:

- Los datos de entrada deben estar en formato numérico. Los atributos categóricos deben ser convertidos a atributos numéricos, empleando alguna de las técnicas existentes.
- Se recomienda que los atributos estén en el rango  $[0, 1]$ , aunque no es imprescindible. En cualquier caso, sí se recomienda que los datos estén en una escala similar, es decir, que no haya atributos en un rango de varias magnitudes superior a otros atributos.

#### 2.3.1. Datos estructurados

Generalmente, en los métodos supervisados partiremos de un conjunto de datos correctamente etiquetados ( $D$ ) en el que distinguimos la siguiente estructura:

$$D_{n,m} = \begin{pmatrix} d_1 = & a_{1,1} & a_{1,2} & \cdots & a_{1,m} & c_1 \\ d_2 = & a_{2,1} & a_{2,2} & \cdots & a_{2,m} & c_2 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ d_n = & a_{n,1} & a_{n,2} & \cdots & a_{n,m} & c_n \end{pmatrix}$$

donde:

- $n$  es número de elementos o instancias,
- $m$  el número de atributos del conjunto de datos (también conocido como dimensionalidad),
- $d_i$  indica la instancia  $i$  del juego de datos,
- $a_{i,j}$  indica el atributo descriptivo  $j$  de la instancia  $i$  y
- $c_i$  indica el atributo objetivo o clase a predecir para la instancia  $d_i$ .

En primer lugar, las redes neuronales, como la mayoría de los algoritmos de aprendizaje automático, prefieren trabajar con **valores numéricos**, así que se debe transformar los atributos categóricos a atributos numéricos.

En segundo lugar, es importante **escalar** los atributos para mejorar el comportamiento y el rendimiento de los modelos de aprendizaje automático.

Hay dos formas básicas de realizar esta operación:

- La normalización (o también conocido como *min-max scaling*) consiste en escalar los valores en el rango  $[0, 1]$ .
- La estandarización se aplica restando el valor medio y dividiendo por la desviación típica o estándar cada uno de los atributos. El resultado no se encuentra limitado en un rango determinado, pero se comporta mejor ante valores extremos (*outliers*).

A diferencia de la normalización, la estandarización no limita los valores a un rango específico, lo que puede ser un problema para algunos algoritmos (por ejemplo, las redes neuronales a menudo esperan un valor de entrada que oscila entre 0 y 1). Sin embargo, la estandarización se ve mucho menos afectada por los valores atípicos o extremos.

### 2.3.2. Imágenes

En el caso de trabajar con imágenes, hay una serie de pasos específicos de preprocesado que debemos llevar a cabo antes de usar los datos en cualquier algoritmo de aprendizaje automático y, en particular, en redes neuronales.

A continuación describimos brevemente los principales:

- Uno de los primeros pasos es garantizar que las imágenes tengan el mismo tamaño y relación de aspecto (*uniform aspect ratio*). La mayoría de los modelos de redes neuronales asumen una imagen de entrada de forma cuadrada, lo que significa que cada imagen debe verificarse si es un cuadrado o no, y recortarse de manera apropiada.
- Una vez que tenemos todas las imágenes en la misma relación de aspecto (ya sean cuadradas o tengan otra relación de aspecto), es el momento de escalar cada imagen de manera adecuada (*image scaling*).
- La normalización de los datos es un paso importante que garantiza que cada parámetro de entrada (píxel, en este caso) tenga una distribución de datos similar. Esto hace que la convergencia sea más rápida mientras se entrena la red.
- Aunque opcional, la reducción de la dimensionalidad es un paso habitual cuando trabajamos con imágenes en color. En algunos casos, se puede optar por colapsar los canales RGB en un solo canal de escala de grises.
- Finalmente, en algunos casos es interesante aumentar el conjunto de datos de forma artificial. Para este fin, se complementa el conjunto de datos con nuevas versiones perturbadas de las imágenes originales. Las perturbaciones típicas incluyen: escalas, rotaciones y otras transformaciones afines. Esta operación puede ayudar a la red a generalizar mejor, evitando el sobreentrenamiento.

### 2.4. Datos de entrenamiento y test

Para validar un modelo o algoritmo de aprendizaje automático es necesario asegurar que este funcionará correctamente para los datos de prueba o test futuros, de forma que capture la esencia del problema a resolver y generalice correctamente. En esencia se trata de evitar que sea dependiente de los datos utilizados durante su entrenamiento, evitando el problema conocido como «sobreentrenamiento» (en inglés, *overfitting*).

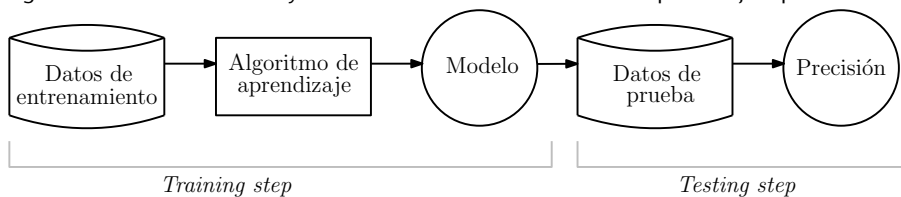
El **sobreentrenamiento** se puede definir, informalmente, como el peligro que corremos al sobreentrenar un modelo de que este acabe respondiendo estric-



tamente a las propiedades del juego de datos de entrenamiento y que sea **incapaz de extrapolarse con niveles de acierto adecuados a otros conjuntos de datos** que puedan aparecer en un futuro.

En la figura 7 queremos subrayar que cuando hablamos de datos de entrenamiento y de test, estamos refiriéndonos en exclusiva a los algoritmos de aprendizaje supervisado. En este escenario es conveniente evaluar los modelos obtenidos sobre datos etiquetados nunca vistos anteriormente y comparar los errores cometidos para cada conjunto.

Figura 7. Proceso de creación y validación de un modelo basado en aprendizaje supervisado



Así, usaremos el conjunto de datos de entrenamiento para crear el modelo supervisado, mientras que el conjunto de datos de test se usará para medir la precisión alcanzada por el modelo. Formará parte del proceso de construcción del modelo la repetición iterativa de entrenamiento y verificación hasta conseguir unos niveles de precisión y de capacidad de predicción aceptables.

Habitualmente los juegos de datos de entrenamiento y de test suelen ser extracciones aleatorias del juego de datos inicial. En función del número de datos disponibles, existen diferentes técnicas para la construcción de los conjuntos de entrenamiento y de prueba. Se trata de un compromiso entre la robustez del modelo construido (a mayor número de datos usados para el entrenamiento, más robusto será el modelo) y su capacidad de generalización (a mayor número de datos usados para la validación, más fiable será la estimación del error cometido).

### 2.4.1. *k-fold cross validation*

Cuando la cantidad de datos disponibles es limitada, una opción interesante y ampliamente utilizada es conocida como *k-fold cross validation*. Esta técnica consiste en realizar una partición aleatoria del conjunto de datos en  $k$  subconjuntos disjuntos del mismo tamaño, empleando  $k - 1$  subconjuntos para entrenar el modelo y el subconjunto restante para evaluarlo. Este proceso se repite  $k$  veces, de tal forma que todos los subconjuntos se emplean una vez en el proceso de evaluación, y se promedia el error estimado.

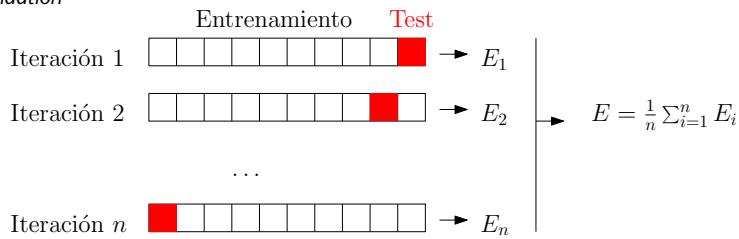
Valores habituales pueden ser  $k = 5$  o  $k = 10$ , aunque no existe ninguna base teórica que sustente dichos valores, sino que es resultado de la experimenta-

#### Lectura complementaria

R. Kohavi. «A study of cross-validation and bootstrap for accuracy estimation and model selection». In Proc. of IJCAI, pp. 1137-1143, San Francisco, CA, USA, 1995. Morgan Kaufmann Publishers Inc.

ción (Kohavi, 1995). La figura 8 muestra el esquema de validación cruzada con  $k = 10$ .

Figura 8. Esquema de división de conjuntos de entrenamiento y test según el método *k-fold cross validation*



Por otra parte, dado que la partición en  $k$  subconjuntos es aleatoria, este proceso podría repetirse un cierto número de ocasiones, promediando todos los errores cometidos (que, de hecho, ya eran promedios del resultado de aplicar *k-fold cross validation*), siempre y cuando se utilice el mismo valor de  $k$ .

Como siempre que hay una partición aleatoria, es importante comprobar que la distribución de los valores de la variable objetivo sea similar, para evitar la creación de modelos muy sesgados. De hecho, la partición aleatoria se debe realizar teniendo en cuenta dicha distribución para evitar el posible sesgo.

## 2.5. Evaluación de modelos

En esta sección revisaremos los principales indicadores o métricas para evaluar los resultados de un proceso de minería de datos, en general, y de un modelo basado en redes neuronales, en particular. El objetivo es cuantificar el grado o valor de «bondad» de la solución encontrada, permitiendo la comparación entre distintos métodos sobre los mismos conjuntos de datos.

Las métricas para realizar este tipo de evaluación dependen, principalmente, del tipo de problema con el que se está lidiando. En este sentido, veremos métricas específicas para problemas de clasificación y regresión.

### 2.5.1. Modelos de clasificación

Las medidas de calidad de modelos de clasificación se calculan comparando las predicciones generadas por el modelo en un conjunto de datos  $D$  con las etiquetas de clase verdaderas de las instancias de este conjunto de datos.

## Matriz de confusión

La matriz de confusión (*Confusion Matrix*, CM) presenta en una tabla una visión gráfica de los errores cometidos por el modelo de clasificación. Se trata de un modelo gráfico para visualizar el nivel de acierto de un modelo de predicción. También es conocido en la literatura como tabla de contingencia o matriz de errores.

Figura 9. Matriz de confusión binaria

|                 |   | Clase predicha |    |
|-----------------|---|----------------|----|
|                 |   | P              | N  |
| Clase verdadera | P | TP             | FN |
|                 | N | FP             | TN |

La figura 9 presenta la matriz de confusión para el caso básico de clasificación binaria. En esencia, esta matriz indica el número de instancias correcta e incorrectamente clasificadas. Los parámetros que nos indica son:

- Verdadero positivo (*True Positive*, TP): número de clasificaciones correctas en la clase positiva (P).
- Verdadero negativo (*True Negative*, TN): número de clasificaciones correctas en la clase negativa (N).
- Falso negativo (*False Negative*, FN): número de clasificaciones incorrectas de clase positiva clasificada como negativa.
- Falso positivo (*False Positive*, FP): número de clasificaciones incorrectas de clase negativa clasificada como positiva.

Como se puede observar, en la diagonal de la matriz aparecen los aciertos del modelo, ya sean positivos o negativos.

## Matriz de confusión para $k$ clases

La matriz de confusión se puede extender a más de dos clases de forma natural, tal y como podemos ver en la figura 10. Esta representación nos permite identificar de forma rápida el número de instancias correctamente clasificadas, que se corresponde con la diagonal de la tabla.

Figura 10. Matriz de confusión para  $k$  clases

|                 |         | Clase predicha |         |       |
|-----------------|---------|----------------|---------|-------|
|                 |         | $C_1$          | $\dots$ | $C_k$ |
| Clase verdadera | $C_1$   |                |         |       |
|                 | $\dots$ |                |         |       |
|                 | $C_k$   |                |         |       |

Por otro lado, en el caso de problemas de clasificación que impliquen más de dos clases, se puede realizar la evaluación de dos formas distintas:

- *1-vs-1* (OvO): midiendo la capacidad de discriminar entre instancias de una clase, considerada la clase positiva, frente a las instancias de otra clase, consideradas negativas.
- *1-vs-All* (OvA): midiendo la capacidad de discriminar entre instancias de una clase, considerada la clase positiva, frente a las instancias de las demás clases, consideradas negativas.

La aproximación de *1-vs-1* nos conduce a una matriz de confusión, con formato  $2 \times 2$ , para cada par de clases existentes. Por otra parte, la aproximación *1-vs-All* produce una matriz de confusión  $2 \times 2$  para cada clase.

### Métricas derivadas de la matriz de confusión

A partir de la matriz de confusión, definimos un conjunto de métricas que permiten cuantificar la bondad de un modelo de clasificación.

- El **error de clasificación** (*Misclassification Error*, ERR) y la **exactitud** (*Accuracy*, ACC) proporcionan información general sobre el número de instancias incorrectamente clasificadas. El error (ecuación 4) es la suma de las predicciones incorrectas sobre el número total de predicciones. Por el contrario, la exactitud (*accuracy*) es el número de predicciones correctas sobre el número total de predicciones, como se puede ver en la ecuación 5.

$$ERR = \frac{FP + FN}{FP + FN + TP + TN} \quad (4)$$

$$ACC = \frac{TP + TN}{FP + FN + TP + TN} = 1 - ERR \quad (5)$$

- En algunos problemas nos puede interesar medir el error en los falsos positivos o negativos. Por ejemplo, **en un sistema de diagnóstico de tumores, nos interesa centrarnos en los casos de tumores malignos que han sido clasificados incorrectamente como tumores benignos**. En estos casos, la **tasa de verdaderos positivos** (*True Positive Rate*, TPR) y la **tasa de falsos positivos** (*False Positive Rate*, FPR), que definimos a continuación, pueden ser muy útiles:

$$TPR = \frac{TP}{FN + TP} \quad (6)$$

$$FPR = \frac{FP}{FP + TN} \quad (7)$$

- La **precisión** (*Precision*, PRE) mide el rendimiento relacionado con las tasas de verdaderos positivos y negativos, tal y como podemos ver a continuación.

$$PRE = \frac{TP}{TP + FP} \quad (8)$$

- El **recall** (*Recall*, REC) y la **sensibilidad** (*Sensitivity*, SEN) se corresponden con la tasa de verdaderos positivos (TPR), mientras que la **especificidad** (*Specificity*, SPE) se define como la tasa de instancias correctamente clasificadas como negativas respecto a todas las instancias negativas.

$$REC = SEN = TPR = \frac{TP}{FN + TP} \quad (9)$$

$$SPE = \frac{TN}{TN + FP} = 1 - FPR \quad (10)$$

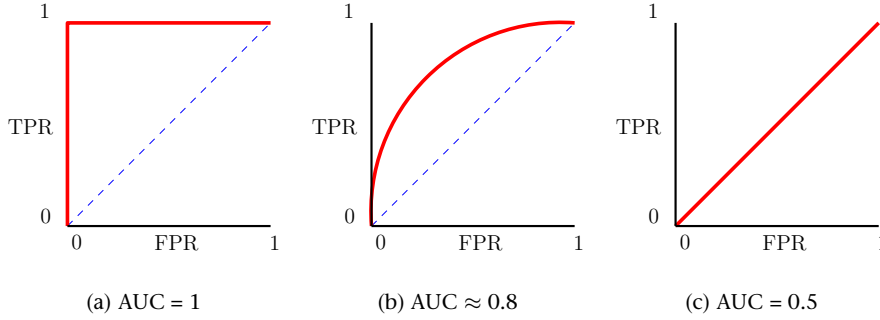
- Finalmente, en la práctica es habitual combinar la precisión y el recall en una métrica llamada **F1** (*F1 score*), que se define de la siguiente forma:

$$F1 = 2 \times \frac{PRE \times REC}{PRE + REC} \quad (11)$$

## Curvas ROC

Una curva ROC (acrónimo de *Receiver Operating Characteristic*) mide el rendimiento respecto a los falsos positivos (FP) y verdaderos positivos (TP). La diagonal de la curva ROC se interpreta como un modelo generado aleatoriamente, mientras que valores inferiores se consideran peores que una estimación aleatoria de los nuevos datos.

Figura 11. Ejemplo de curvas ROC



En esta métrica, un clasificador perfecto ocuparía la posición superior izquierda de la gráfica, con una tasa de verdaderos positivos (TPR) igual a 1 y una tasa de falsos positivos (FPR) igual a 0. A partir de la curva ROC se calcula el **área bajo la curva** (*Area Under the Curve*, AUC) que permite caracterizar el rendimiento de un modelo de clasificación. La figura 11 ejemplifica un rendimiento excelente, bueno y malo (aleatorio) de una curva ROC.

### 2.5.2. Modelos de regresión

La evaluación de modelos de regresión comparte el mismo principio que la evaluación de modelos de clasificación, en el sentido de que se comparan los valores predichos con los valores reales de las instancias del conjunto de datos. En esta sección veremos algunas de las métricas más empleadas para evaluar el rendimiento de un modelo de regresión.

- El **error absoluto medio** (*Mean Absolute Error*, MAE) es la métrica más simple y directa para la evaluación del grado de divergencia entre dos conjuntos de valores, representado por la ecuación:

$$MAE = \frac{1}{|D|} \sum_{d \in D} |f(d) - h(d)| \quad (12)$$

donde  $D$  es el conjunto de instancias,  $h : X \rightarrow \mathbb{R}$  es la función del modelo y  $f : X \rightarrow \mathbb{R}$  es la función objetivo con las etiquetas correctas de las instancias.

En este caso, todos los residuos tienen la misma contribución al error absoluto final.

- El **error cuadrático medio** (*Mean Square Error*, MSE) es, probablemente, la métrica más empleada para la evaluación de modelos de regresión. Se calcula de la siguiente forma:

$$MSE = \frac{1}{|D|} \sum_{d \in D} (f(d) - h(d))^2 \quad (13)$$

Utilizando esta métrica se penaliza los residuos grandes. En este sentido, si el modelo aproxima correctamente a gran parte de las instancias del conjunto de datos, pero comete importantes errores en unos pocos, la penalización en esta métrica será muy superior a la indicada si se emplea la métrica anterior (MAE).

- La **raíz cuadrada del error cuadrático medio** (*Root Mean Square Error*, RMSE) se define aplicando la raíz cuadrada al error cuadrático medio (MSE). Tiene las mismas características que este, pero aporta la ventaja adicional de que el error se expresa en la misma escala que los datos originales. En el caso del MSE no era así, ya que la diferencia de valores son elevados al cuadrado antes de realizar la suma ponderada. En algunos casos esta diferencia puede ser importante, mientras que irrelevante en otros.

$$RMSE = \sqrt{MSE} = \sqrt{\frac{1}{|D|} \sum_{d \in D} (f(d) - h(d))^2} \quad (14)$$

- Finalmente, en algunos modelos de regresión se pueden tolerar diferencias importantes entre los valores predichos y verdaderos, siempre que el modelo tenga un comportamiento general similar a los valores verdaderos, prestando especial atención a la monotonidad. En estos casos, las métricas vistas anteriormente pueden no ser representativas del rendimiento de un modelo de regresión. Por el contrario, los **índices de correlación** lineal o de rango, como por ejemplo **Pearson o Spearman**, pueden ser una buena métrica para **medir la bondad del modelo generado**.

### 3. Apéndice A: Notación

Generalmente, en los métodos supervisados partiremos de un conjunto de datos correctamente etiquetados ( $D$ ) en el que distinguimos la siguiente estructura:

$$D_{n,m} = \begin{pmatrix} d_1 = & a_{1,1} & a_{1,2} & \cdots & a_{1,m} & c_1 \\ d_2 = & a_{2,1} & a_{2,2} & \cdots & a_{2,m} & c_2 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ d_n = & a_{n,1} & a_{n,2} & \cdots & a_{n,m} & c_n \end{pmatrix}$$

donde:

- $n$  es el número de elementos o instancias,
- $m$  es el número de atributos del conjunto de datos o también su dimensionalidad,
- $d_i$  indica cada una de las instancias del juego de datos,
- $a_{i,j}$  indica cada uno de los atributos descriptivos y
- $c_i$  indica el atributo objetivo o clase a predecir para cada elemento.

En el caso de los métodos no supervisados, el conjunto de datos sigue el mismo patrón y notación, excepto para el atributo de clase ( $c_i$ ) que no existe (o no se emplea) en el caso de los conjuntos no etiquetados.

La tabla 1 muestra un resumen de la notación empleada en los distintos módulos didácticos.

Tabla 1. Resumen de la notación básica

| Símbolo   | Descripción                                                          |
|-----------|----------------------------------------------------------------------|
| $D_{n,m}$ | Conjunto de datos                                                    |
| $n$       | Número de instancias                                                 |
| $m$       | Número de atributos                                                  |
| $k$       | Número de clases                                                     |
| $d_i$     | Instancia $i$ del conjunto de datos $D$                              |
| $a_j$     | Conjunto del atributo $j$ en $D$                                     |
| $a_{i,j}$ | Valor del atributo $j$ en la instancia $i$ del conjunto de datos $D$ |
| $c_i$     | Clase de la instancia $d_i$                                          |



## Resumen

En este texto hemos presentado una introducción al aprendizaje profundo (*deep learning*, en inglés), definiendo qué es el aprendizaje profundo y cuáles son sus principales características y diferencias con otros tipos de aprendizaje automático. Seguidamente, hemos realizado un breve repaso histórico sobre las redes neuronales, que constituyen la base de los modelos de aprendizaje profundo.

En el segundo bloque de este módulo didáctico se han repasado algunos conceptos básicos sobre aprendizaje automático que son necesarios para comprender y profundizar en los modelos de aprendizaje profundo. En concreto, hemos revisado los conceptos relacionados con la tipología de métodos y tareas en el aprendizaje automático, así como los principios de preprocesamiento de datos, generación de los conjuntos de entrenamiento y test, y métricas de evaluación de resultados.

## Glosario

**aprendizaje automático** *m* El aprendizaje automático (más conocido por su denominación en inglés, *machine learning*) es el conjunto de técnicas, métodos y algoritmos que permiten a una máquina aprender de manera automática en base a experiencias pasadas.

**aprendizaje no supervisado** *m* El aprendizaje no supervisado se basa en el descubrimiento de patrones, características y correlaciones en los datos de entrada, sin intervención externa.

**aprendizaje supervisado** *m* El aprendizaje supervisado se basa en el uso de un componente externo, llamado supervisor, que compara los datos obtenidos por el modelo con los datos esperados por este, y proporciona retroalimentación al modelo para que vaya ajustándose y mejorando las predicciones.

## Bibliografía

Bengio, Y., Goodfellow, I., Courville, A. (2016). *Deep Learning*. Cambridge, MA: MIT Press.

Casas Roma, J., Lozano Bagén, T., Bosch Rué, A. (2020). *Deep learning: Principios y fundamentos*. Barcelona: Editorial UOC.

Gironés Roig, J., Casas Roma, J., Minguillón Alfonso, J., Caihuelas Quiles, R. (2017). *Minería de datos: Modelos y algoritmos*. Barcelona: Editorial UOC.

Haykin, S. O. (2009). *Neural Networks and Learning Machines*. 3ª edición. Pearson.